

MiniLibX Python Manual

Nora de Fitero Teixeira (dde-fite)



Abstract

This documentation is a PORT of the ORIGINAL MiniLibX docs. It describes the Python package that provides access to the MiniLibX graphics library. It allows creating windows, drawing pixels, handling images, and capturing keyboard and mouse input through a thin wrapper over the original C API, keeping function names and behavior as close as possible to the native MiniLibX library.

Acknowledgements

The original MiniLibX documentation was created by Olivier Crouzet under the MIT license. This is a derivative work based on his work, created on a non-profit basis with the aim of sharing knowledge among the 42 student community. This derivative work is published under the MIT license by Nora de Fitero Teixeira. Copies of the licenses can be found in the GitHub repository for this documentation.

Table of Contents

Introduction	2
How a graphics server works	2
Historical X-Window concept	2
Modern graphical approach	2
Getting started	3
Clone the source code	3
Requirements	3
Arch Linux	3
Debian/Ubuntu	3
Installation	3
Fallback	4
Example of use	4
Behind the Scenes	5
Initialization and cleanup: <code>mlx_init()</code>, <code>mlx_release()</code>	5
Synopsis	5
Description	6
Return values	6
Managing windows: <code>mlx_new_window()</code>, <code>mlx_clear_window</code>, <code>mlx_destroy_window</code>	6
Synopsis	6
Description	6
Return values	7
Drawing inside windows: <code>mlx_pixel_put()</code>, <code>mlx_string_put()</code>	7
Synopsis	7
Description	7
Color management	8
Manipulating images: <code>mlx_new_image()</code>, <code>mlx_get_data_addr()</code>, <code>mlx_put_image_to_window()</code>, <code>mlx_xpm_file_to_image()</code>, <code>mlx_png_file_to_image()</code>, <code>mlx_destroy_image()</code>	8
Synopsis	8
Description	9
Storing colours inside images	9
XPM and PNG images	10

Return values	10
Handle events: <code>mlx_loop()</code>, <code>mlx_key_hook()</code>, <code>mlx_mouse_hook()</code>, <code>mlx_expose_hook()</code>, <code>mlx_loop_hook()</code>, <code>mlx_loop_hook()</code>	10
Synopsis	10
Events	11
Description	11
Going further with events	12
Extra functions	12
Synopsis	12
Mouse extra functions	12
Keyboard extra functions	12
Screen extra function	13
Flush and sync functions	13
Got any suggestions?	13

Introduction

The MiniLibX Python Wrapper allows you to create graphical software easily without any knowledge of X-Window/Wayland/Vulkan on Unix/Linux, or AppKit on macOS. It provides:

- Window creation and management
- Pixel-level drawing
- Image manipulation for faster rendering
- Keyboard and mouse input handling
- PNG and XPM image loading

How a graphics server works

This library interacts with the underlying graphics system of your operating system. Before diving into usage, it's helpful to understand how graphics servers manage windows and handle user input.

Historical X-Window concept

X-Window is a network-oriented graphical system for Unix. It is based on two main parts:

- On one side, your software wants to draw something on the screen and or get keyboard & mouse entries.
- On the other side, the X-Server manages the screen, keyboard and mouse (It is often referred to as a "display").

A network connection must be established between these two entities to send drawing orders (from the software to the X-Server), and keyboard/mouse events (from the X-Server to the software).

Nowadays, most of the time, both run on the same computer.

Modern graphical approach

Modern computers come with a powerful GPU that is directly accessed by applications. Along GPU libraries like Vulkan or OpenGL, the Wayland protocol ensure communication with the compositor program that manages the various windows on screen and the user input events. For your own application:

- The Vulkan or OpenGL library allow you to directly draw any content into your window.

- The Wayland compositor handles the place of your window on screen and send you back the keyboard and mouse inputs from the user.

Unfortunately, this gain of graphical power through GPU access removes the networking aspects that exist with X-Window. It is not possible for a program to access a remote GPU and show its window on a remote display. But current software architectures are more likely based on a local display application that gets data in JSON through a web API.

Getting started

Clone the source code

```
git clone https://github.com/42school/mlx_CLXV.git
```

Requirements

Arch Linux

```
sudo pacman -S libxcb xcb-util-keysyms zlib libbsd vulkan-icd-loader  
↳ vulkan-tools shaderc
```

Debian/Ubuntu

```
sudo apt install libxcb libxcb-keysyms libvulkan libz libbsd glslc
```

Installation

First compile MiniLibX.

```
make install
```

Create a virtual environment with your preferred manager and open it:

- For bash/zsh:

```
python -m venv .venv  
source .venv/bin/activate
```

- For fish:

```
python -m venv .venv
source .venv/bin/activate.fish
```

And install the package:

```
pip install mlx_CLXV-2.2-py3-none-any.whl
```

Fallback

As a fallback, you can use the packages distributed in the intranet **mlx-2.2-py3-ubuntu-any.whl** and **mlx-2.2-py3-fedora-any.whl** (accessed on January 29, 2026).

In case you get an error when installing the distributed packages, try changing the name to **mlx-2.2-py3-none-any.whl**.

Example of use

This small Python script displays a small black window with text. It will also print the screen dimensions to stdout and listen for user clicks.

We will explain the functions used in this example later.

```
from mlx import Mlx

def mymouse(button, x, y, mystuff):
    print(f"Got mouse event! button {button} at {x},{y}.")

def mykey(keynum, mystuff):
    print(f"Got key {keynum}, and got my stuff back:")
    print(mystuff)
    if keynum == 32:
        m.mlx_mouse_hook(win_ptr, None, None)

m = Mlx()
mlx_ptr = m.mlx_init()
win_ptr = m.mlx_new_window(mlx_ptr, 200, 200, "test")
m.mlx_clear_window(mlx_ptr, win_ptr)
m.mlx_string_put(mlx_ptr, win_ptr, 20, 20, 255, "Hello PyMlx!")
(ret, w, h) = m.mlx_get_screen_size(mlx_ptr)
print(f"Got screen size: {w} x {h} .")

stuff = [1, 2]
m.mlx_mouse_hook(win_ptr, mymouse, None)
```



```
m.mlx_key_hook(win_ptr, mykey, stuff)
```

```
m.mlx_loop(mlx_ptr)
```

Behind the Scenes

When an instance of the Mlx class is created, the first thing it does is construct the path to the C library called libmlx.so.

```
def __init__(self):
    module_dir = os.path.dirname(os.path.abspath(__file__))
    self.so_file = os.path.join(module_dir, "libmlx.so")
    #...
```

- `__file__` is a special Python variable that contains the path to the file where this code is executed.
- `os.path.dirname` extracts the path from **file** and uses `os.path.join` to create the path to the library.

It then declares the `mlx_func` variable, which acts as a bridge between Python and C. Using the `CDLL` function from Python's `ctypes` module, it loads the library and calls the original functions.

```
def __init__(self):
    # ...
    self.mlx_func = CDLL(self.so_file)
    # ...
```

For each C function available in the Python wrapper, there is a declaration within the Mlx class:

```
def mlx_init(self):
    self.mlx_func.mlx_init.restype = c_void_p
    return self.mlx_func.mlx_init()
```

You can see how it calls `mlx_func.mlx_init()`. This `mlx_init()` is already the original C function. It is necessary to specify the data type returned by the function with `mlx_init.restype`, which in this case is `c_void_p` (equivalent to `void *`).

All of this is passed to `CDLL`, which, using the previously loaded library, will execute the function and return whatever the function returns.

Initialization and cleanup: `mlx_init()`, `mlx_release()`

Synopsis

```
from mlx import Mlx
```

```
def mlx_init() -> int: # void *
```

```
def mlx_release() -> int: # void *
```

Description

First of all, you need to initialize the connection between your software and the graphic and user sub-systems. Once this completed, you'll be able to use other MiniLibX functions to send and receive the messages from the display, like "I want to draw a yellow pixel in this window" or "did the user hit a key?".

The `mlx_init` function will create this connection. No parameters are needed, and it will return a `void *` identifier, used for further calls to the library routines. The `mlx_release` function can be used at the end of the program to disconnect from the graphic system and release resources.

If **`mlx_init()`** fails to set up the connection to the display, it will return `None`.

Return values

If **`mlx_init()`** set up the connection to the display correctly, it will return an **`int as a pointer`**; otherwise, it returns **`None`**.

Managing windows: `mlx_new_window()`, `mlx_clear_window`, `mlx_destroy_window`

Synopsis

```
def mlx_new_window(mlx_ptr: int, width: int, height: int, title: str )  
↳ -> int: # void *
```

```
def mlx_clear_window(mlx_ptr: int, win_ptr: int) -> int:
```

```
def mlx_destroy_window(mlx_ptr: int, win_ptr: int ) -> int:
```

Description

The **`mlx_new_window ()`** function creates a new window on the screen, using the *width* and *height* parameters to determine its size, and *title* as the text that should be displayed in the window's title

bar.

The *mlx_ptr* parameter is the connection identifier returned by **mlx_init**() (see the **mlx** man page). **mlx_new_window**() returns a *void ** window identifier that can be used by other MiniLibX calls.

Note that the MiniLibX can handle an arbitrary number of separate windows.

mlx_clear_window() and **mlx_destroy_window**() respectively clear (in black) and destroy the given window. They both have the same parameters: *mlx_ptr* is the screen connection identifier, and *win_ptr* is a window identifier.

Return values

If **mlx_new_window()** fails to create a new window (whatever the reason), it will return NULL, otherwise a non-null pointer is returned as a window identifier.

mlx_clear_window and **mlx_destroy_window** return nothing.

Drawing inside windows: **mlx_pixel_put()**, **mlx_string_put()**

Synopsis

```
def mlx_pixel_put(mlx_ptr: int, win_ptr: int, x: int, y: int, color:
    ↪ int) -> int:

def mlx_string_put(mlx_ptr: int, win_ptr: int, x: int, y: int, color:
    ↪ int, string: str) -> int:
```

Description

The **mlx_pixel_put**() function draws a defined pixel in the window *win_ptr* using the (*x*, *y*) coordinates, and the specified *color*.

The origin (0,0) is the upper left corner of the window, the *x* and *y* axis respectively pointing right and down. The connection identifier, *mlx_ptr*, is needed (see the **mlx** man page).

Parameters for **mlx_string_put**() have the same meaning. Instead of a simple pixel, the specified *string* will be displayed at (*x*, *y*).

Both functions will discard any display outside the window. This makes **mlx_pixel_put** slow. Consider using images instead.

Color management

The *color* parameter has an unsigned integer type. The displayed colour needs to be encoded in this integer, following a defined scheme. All displayable colours can be split in 3 basic colours: red, green and blue. Three associated values, in the 0-255 range, represent how much of each colour is mixed up to create the original colour. The fourth byte represent transparency, where 0 is fully transparent and 255 opaque.

Theses four values must be set inside the unsigned integer to display the right colour. The bytes of this integer are filled as shown in the picture below:

```
| B | G | R | A |   colour integer
+---+---+---+---+
```

While filling the integer, make sure you avoid endian problems.

Example: the "blue" byte will be the least significant byte inside the integer on a little endian machine.

Manipulating images: `mlx_new_image()`, `mlx_get_data_addr()`, `mlx_put_image_to_window()`, `mlx_xpm_file_to_image()`, `mlx_png_file_to_image()`, `mlx_destroy_image()`

Synopsis

```
def mlx_new_image(mlx_ptr: int, width: int, height: int) -> int: # void
↳ *

def mlx_get_data_addr(img_ptr: int, bits_per_pixel: int, size_line:
↳ int, format: int) -> tuple[memoryview, int, int, int]:

def mlx_put_image_to_window(mlx_ptr: int, win_ptr: int, img_ptr: int,
↳ x: int, y: int) -> int:

def mlx_xpm_file_to_image(mlx_ptr: int, filename: str, width: int,
↳ height: int) -> int: # void *

def mlx_png_file_to_image(mlx_ptr: int, filename: str, width: int,
↳ height: int) -> int: # void *

def mlx_destroy_image(mlx_ptr: int, img_ptr: int) -> int:
```

Description

mlx_new_image() creates a new image in memory. It returns an *int pointer* needed to manipulate this image later. It only needs the size of the image to be created, using the *width* and *height* parameters, and the *mlx_ptr* connection identifier.

The user can draw inside the image (see below), and can dump the image inside a specified window at any time to display it on the screen. This is done using **mlx_put_image_to_window()**. Three identifiers are needed here, for the connection to the display, the window to use, and the image (respectively *mlx_ptr*, *win_ptr* and *img_ptr*). The (x, y) coordinates define where the image should be placed in the window.

mlx_get_data_addr() returns information about the created image, allowing a user to modify it later. The *img_ptr* parameter specifies the image to use. The three next parameters should be the addresses of three different valid unsigned integers. *bits_per_pixel* will be filled with the number of bits needed to represent a pixel colour (also called the depth of the image). *size_line* is the number of bytes used to store one line of the image in memory. This information is needed to move from one line to another in the image. *format* tells you how each pixel colour in the image is structured.

Currently only 2 values are defined:

0 means format B8G8R8A8

1 means format A8R8G8B8

mlx_get_data_addr returns an *int pointer* to the address address that represents the beginning of the memory area where the image is stored. From this address, the first *bits_per_pixel* bits represent the colour of the first pixel in the first line of the image. The second group of *bits_per_pixel* bits represent the second pixel of the first line, and so on. Add *size_line* to the address to get the beginning of the second line. You can reach any pixels of the image that way.

mlx_destroy_image destroys the given image (*img_ptr*).

Storing colours inside images

Depending on the graphic system, the number of bits used to store a pixel colour used to be different from one hardware to another. Today, the way the user usually represents a colour, in the ARGB mode, almost always matches the hardware capabilities on modern computers.

Keep in mind that packing the 4-byte ARGB into an unsigned int depends on the local computer's endian. Adjust your code accordingly.

XPM and PNG images

The **mlx_xpm_file_to_image()** and **mlx_png_file_to_image()** functions will create a new image the same way. They will fill it using the specified *xpm_data* or *filename*, depending on which function is used. Note that MiniLibX does not use the standard Xpm and png libraries to deal with xpm and png images.

You may not be able to read all types of xpm and png images. It however handles transparency.

mlx_xpm_to_image() is not implemented in the Python wrapper as it is considered not useful in this context.

Return values

The three functions that create images, **mlx_new_image()**, **mlx_xpm_file_to_image()** and **mlx_png_file_to_image()**, will return NULL if an error occurs.

Otherwise they return a non-null pointer as an image identifier.

Handle events: **mlx_loop()**, **mlx_key_hook()**, **mlx_mouse_hook()**, **mlx_expose_hook()**, **mlx_loop_hook()**, **mlx_loop_hook()**

Synopsis

```
def mlx_loop(mlx_ptr: int) -> int:

def mlx_key_hook(win_ptr: int, callback: Callable[Any], param: Any) ->
    int:

def mlx_mouse_hook(win_ptr: int, callback: Callable[Any], param: Any)
    -> int:

def mlx_expose_hook(win_ptr: int, callback: Callable[Any], param: Any)
    -> int:

def mlx_loop_hook(mlx_ptr: int, callback: Callable[Any], param: Any) ->
    int:

def mlx_loop_exit(mlx_ptr: int) -> None:
```

Events

The graphical system is bi-directional. On one hand, the program sends orders to the screen to display pixels, images, and so on. On the other hand, it can get information from the keyboard and mouse associated to the screen. To do so, the program receives “events” from the keyboard or the mouse.

Description

To receive events, you must use **mlx_loop** (). This function never returns, unless **mlx_loop_exit** is called. It is an infinite loop that waits for an event, and then calls a user-defined function associated with this event. A single parameter is needed, the connection identifier *mlx_ptr*.

You can assign different functions to the three following events: - A key is released - The mouse button is pressed - A part of the window should be re-drawn (this is called an “expose” event, and it is your program’s job to handle it in the Unix/Linux X11 environment, but at the opposite it never happens on Unix/Linux Wayland-Vulkan nor on MacOS).

Each window can define a different function for the same event.

The three functions **mlx_key_hook** (), **mlx_mouse_hook** () and **mlx_expose_hook** () work exactly the same way. *callback* is a reference to the function that is invoked when an event occurs. This assignment is specific to the window defined by the *win_ptr* identifier. The *param* address will be passed back to your function every time it is called, and should be used to store the parameters it might need.

The syntax for the **mlx_loop_hook** () function is similar to the previous ones, but the given function will be called when no event occurs, and is not bound to a specific window.

When it catches an event, the MiniLibX calls the corresponding function with fixed parameters:

```
expose_hook(void *param);
key_hook(unsigned int keycode, void *param);
mouse_hook(unsigned int button, unsigned int x, unsigned int y, void *param);
loop_hook(void *param);
```

These function names are arbitrary. They here are used to distinguish parameters according to the event. These functions are NOT part of the MiniLibX.

param is the address specified in the *mlx_*_hook* calls. This address is never used nor modified by the MiniLibX. On key and mouse events, additional information is passed: *keycode* tells you which key is pressed (just try to find out:) , (*x* , *y*) are the coordinates of the mouse click in the window, and *button* tells you which mouse button was pressed.

Going further with events

The MiniLibX provides a much generic access to other available events. The *mlx.h* include define **mlx_hook()** in the same manner *mlx_*_hook* functions work. The event and mask values will be taken from the historical X11 include file "X.h". Some Wayland and MacOS events are mapped to these values when it makes sense, and the mask may not be used in some configurations.

See source code of the MiniLibX to find out how it will call your own function for a specific event.

Extra functions

Synopsis

```
def mlx_mouse_hide(mlx_ptr: int) -> int:

def mlx_mouse_show(mlx_ptr: int) -> int:

def mlx_mouse_move(mlx_ptr: int, x: int, y: int) -> int:

def mlx_mouse_get_pos(win_ptr: int) -> tuple(int, int, int):

def mlx_do_key_autorepeatoff(mlx_ptr: int) -> int:

def mlx_do_key_autorepeaton(mlx_ptr: int) -> int:

def mlx_get_screen_size(mlx_ptr: int) -> tuple(int, int, int):

def mlx_do_sync(mlx_ptr: int) -> int:

def mlx_sync(mlx_ptr: int, cmd: int, img_or_win_ptr: int) -> int:
```

Mouse extra functions

It is possible to show / hide the mouse, and get its current position without user click or force its position inside a window.

Keyboard extra functions

The auto-repeat mode of the keyboard can be controlled. By default, auto-repeat is on: multiple “key pressed” events are generated every second until the key is released.

Screen extra function

It is possible to retrieve the size of the current screen, even before the first window is created.

Flush and sync functions

The **mlx_do_sync** function will flush the pending commands to the graphic subsystems, ensuring nothing is cached on your software's side. On return, there is no guarantee that your commands have been processed.

With **mlx_sync** you have more detailed control over the synchronisation mechanisms. Three different commands are available: SYNC_IMAGE_WRITABLE = 1 SYNC_WIN_FLUSH = 2 SYNC_WIN_COMPLETED = 3 The third parameter *param* can be either the image identifier or the window identifier.

Got any suggestions?

If you find any errors or have any new ideas for improving this repository, feel free to open an Issue or Pull Request, or contact me at my email address: nora@defitero.com